MASI — Mini Projet

Application JavaFX de Dessin Géométrique

Salha HENTATI
Chaima MAATAR

Année Universitaire 2025 / 2026

1. Vue d'ensemble de l'architecture

L'application est une zone de dessin interactive permettant à l'utilisateur de tracer des rectangles, cercles et lignes, de les colorer, de sauvegarder ses dessins en base SQLite et de calculer des plus courts chemins dans un graphe visuel.

L'architecture suit une organisation en quatre couches :

Couche	Composants
Présentation	MainController (FXML) · DrawingApp · main.fxml
Application	DrawingCanvas · Command · Historique
Domaine	Forme · Dessin · Decorator · Factory · Adapter
Infrastructure	FormeDAOImpl · DatabaseConnection · Logger · LogStrategy · AlgorithmePCC · GrapheContext

Répartition des patrons par couche

Couche	Patron(s) utilisé(s)
Infrastructure	Singleton, Strategy (Log), Strategy (PCC)
Domaine	Decorator, Factory Method, Adapter, Memento
Application	Command, Memento (Caretaker), Observer (Subject)
Présentation	Observer (Client), Command (Invoker)

2. Description des packages et fichiers

Tout le code source se trouve sous src/main/java/masi/. Voici la description de chaque package :

Package masi.app — Point d'entrée

Fichier	Rôle
DrawingApp.java	Classe principale (main). Lance JavaFX, initialise le Singleton BD, charge le FXML.
DrawingCanvas.java	Zone de dessin. Joue trois rôles : Receiver (Command), Subject (Observer), Client (Adapter).
MainController.java	Contrôleur FXML. Joue le rôle d'Invoker (Command) et orchestre tous les patrons.

Package masi.model — Entités métier

Fichier	Rôle
Forme.java	Classe abstraite racine. Originator du patron Memento. Définit l'état commun.
FormeRectangle.java	Forme concrète Rectangle. Encapsule un nœud JavaFX Rectangle.
FormeCercle.java	Forme concrète Cercle. Encapsule un nœud JavaFX Circle.
FormeLigne.java	Forme concrète Ligne. Encapsule un nœud JavaFX Line.
Dessin.java	Entité agrégat. Contient un nom, une date et une liste de Forme.

Package masi.decorator — Patron Decorator

Fichier	Rôle
FormeDecorator.java	Décorateur abstrait. Hérite de Forme et délègue vers une Forme wrappée.
FormeColoree.java	Décorateur concret — applique une couleur personnalisée.
FormeBordure.java	Décorateur concret — applique une bordure avec épaisseur.

Package masi.factory — Patron Factory Method

Fichier	Rôle
FormeFactory.java	Creator abstrait. Déclare la factory method creerForme().
FormesConcretes.java	Contient RectangleFactory, CercleFactory, LigneFactory.
FormeFactoryProvider.java	Point d'entrée public — sélectionne et invoque la factory appropriée.

Package masi.command — Patron Command

Fichier	Rôle
Command.java	Interface. Déclare execute() et undo().
AjouterFormeCommand.java	Commande concrète — ajoute une forme au canvas.
SupprimerFormeCommand.java	Commande concrète — supprime une forme du canvas.

Package masi.memento — Patron Memento

Fichier	Rôle
Historique.java	Caretaker. Maintient deux piles de Forme.Memento pour undo/redo.

La classe Forme.Memento (inner class privée de Forme) joue le rôle de Memento.

Package masi.observer — Patron Observer

Fichier	Rôle
Observable.java	Interface Subject. Déclare addObs, removeObs, notifyAllObs.
Observer.java	Interface Observer. Déclare update(String message).

Package masi.singleton — Patron Singleton

Fichier	Rôle
DatabaseConnection.java	Singleton — unique connexion JDBC SQLite dans l'application.
Logger.java	Singleton + Observer + Strategy Context — journalisation centralisée.

Package masi.strategy — Patron Strategy (Journalisation)

Fichier	Rôle
LogStrategy.java	Interface Strategy. Déclare log(String message).
ConsoleLogStrategy.java	Stratégie concrète — écrit sur System.out.
FichierLogStrategy.java	Stratégie concrète — écrit dans log.txt.
DatabaseLogStrategy.java	Stratégie concrète — insère dans la table LOG SQLite.

Package masi.adapter — Patron Adapter

Fichier	Rôle
FormeMetier.java	Interface Target. Contrat d'accès aux formes indépendant de JavaFX.
FormeJFXAdapter.java	Adapter. Traduit les appels de FormeMetier vers Forme (Adaptee).

Package masi.graphe — Patron Strategy (PCC)

Fichier	Rôle
AlgorithmePCC.java	Interface Strategy. Déclare calculer(adjacences, depart, arrivee).
DijkstraStrategy.java	Stratégie concrète — chemin de poids minimal (file de priorité).
BFSStrategy.java	Stratégie concrète — chemin avec le moins d'arêtes (BFS, file FIFO).
GrapheContext.java	Context Strategy. Maintient la liste d'adjacence et délègue le calcul.
Arete.java	Entité. Représente une arête avec destination et poids.

3. Patron 1 — Singleton

Contexte et problème résolu

L'application nécessite une unique connexion à la base de données SQLite partagée entre le DAO et la stratégie de log. Créer plusieurs connexions entraînerait des incohérences et des verrous. De même, le Logger doit être unique pour que toute l'application écrive dans le même journal.

> Le patron Singleton garantit qu'une classe n'a qu'une seule instance et fournit un point d'accès global à cette instance.

Diagramme UML

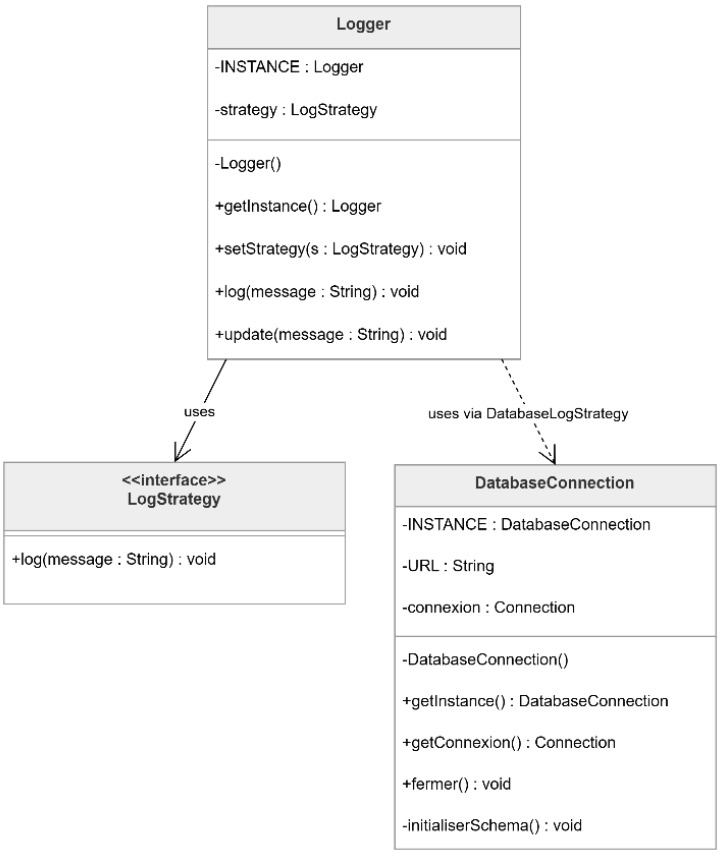


Figure 1: Diagramme Singleton — Connexion BD et Logger centralisés

4. Patron 2 — Observer

Contexte et problème résolu

Quand l'utilisateur dessine ou supprime une forme, plusieurs composants doivent être informés : le Logger pour écrire dans le journal, et la zone de texte txtLog de l'IHM pour afficher le message à l'écran. Ces composants ne doivent pas être couplés directement à DrawingCanvas.

> Le patron Observer définit une dépendance un-à-plusieurs : quand un objet change d'état, tous ses dépendants sont notifiés automatiquement.

Diagramme UML

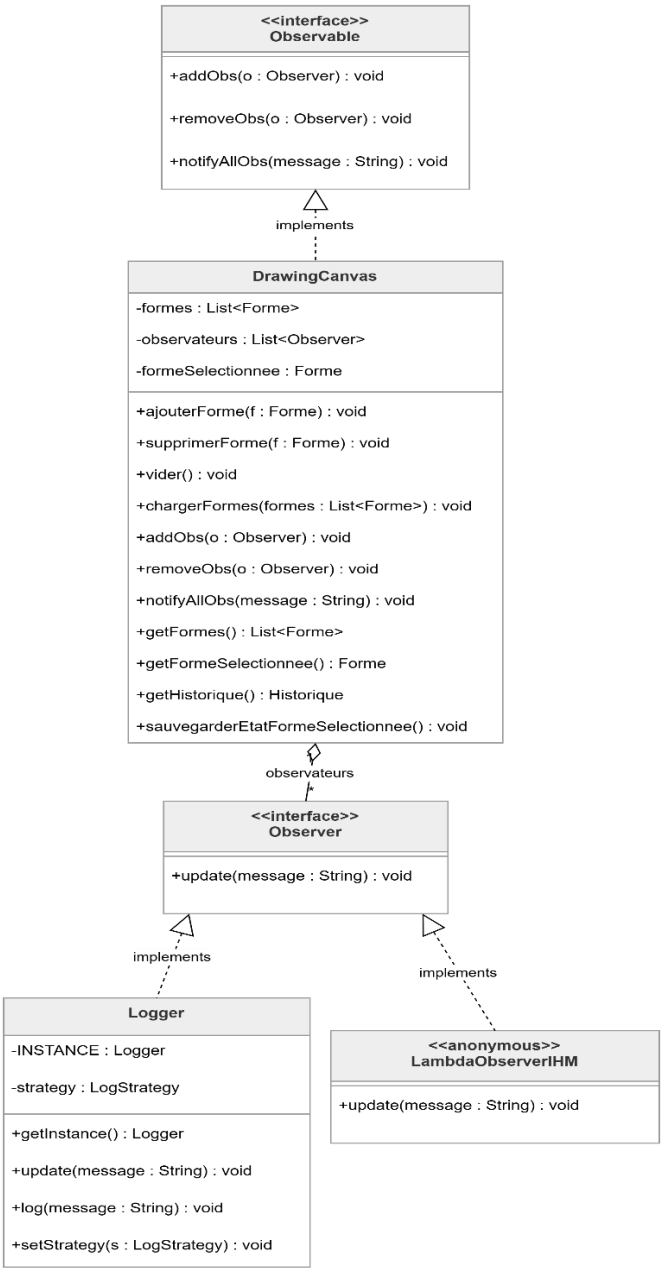


Figure 2: Diagramme Observer — Notification IHM et journalisation

5. Patron 3 — Strategy — Journalisation

Contexte et problème résolu

L'application doit pouvoir journaliser les événements de trois façons différentes : dans la console, dans un fichier texte, ou en base de données. L'utilisateur peut changer le mode à l'exécution depuis l'IHM. Sans Strategy, il faudrait un if/else géant dans le Logger, et ajouter un nouveau mode obligerait à modifier le Logger.

> Le patron Strategy définit une famille d'algorithmes, encapsule chacun dans une classe séparée, et les rend interchangeables.

Diagramme UML

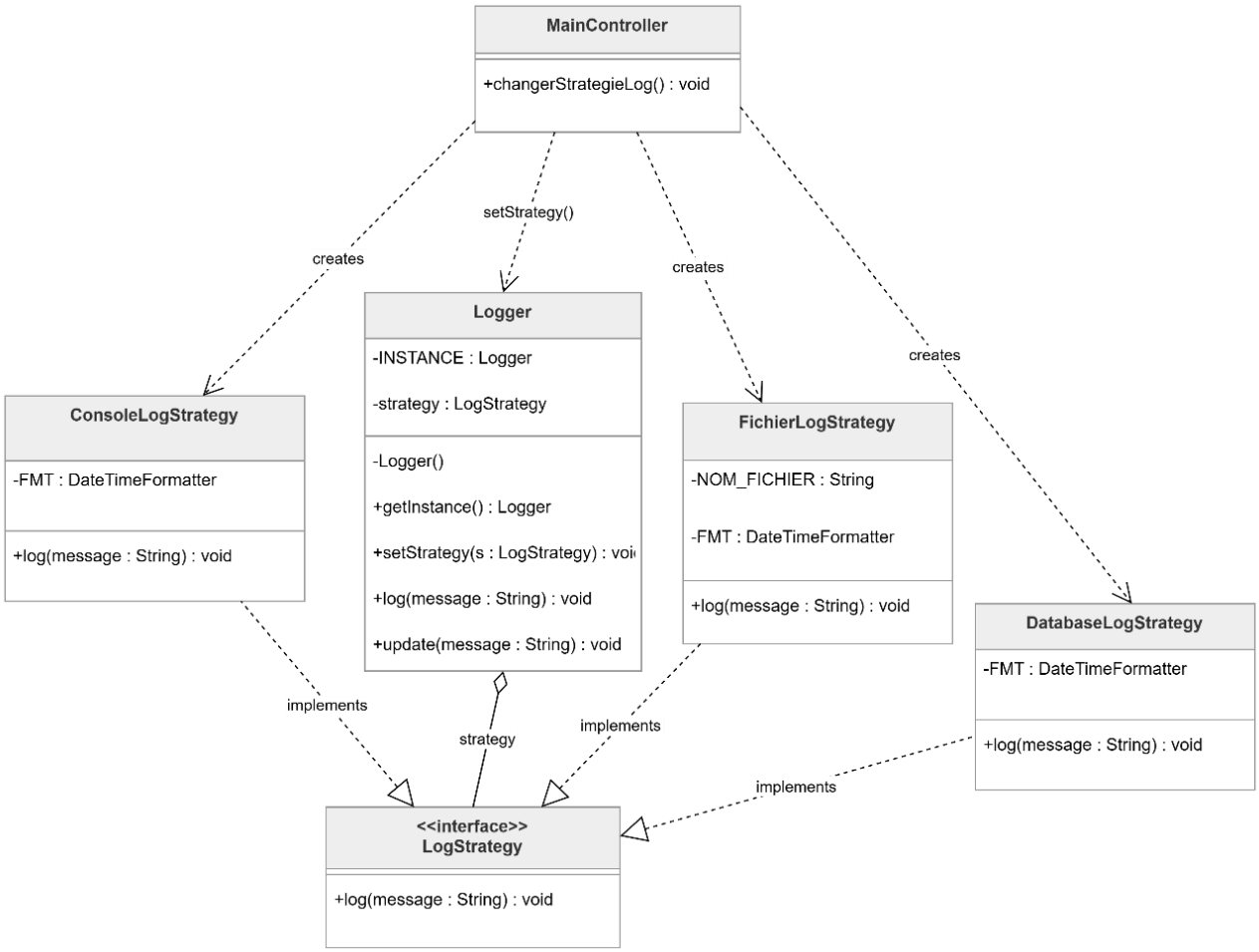


Figure 3: Diagramme Strategy — Journalisation interchangeable (Console / Fichier / BD)

6. Patron 4 — Strategy — Algorithmes Plus Court Chemin (PCC)

Contexte et problème résolu

Le module graphe doit calculer le plus court chemin entre deux nœuds selon deux algorithmes différents : Dijkstra (minimise la somme des poids) et BFS (minimise le nombre d'arêtes). L'utilisateur choisit l'algorithme en runtime depuis un ComboBox.

> Seconde application du patron Strategy, cette fois pour des algorithmes de graphe.

Diagramme UML

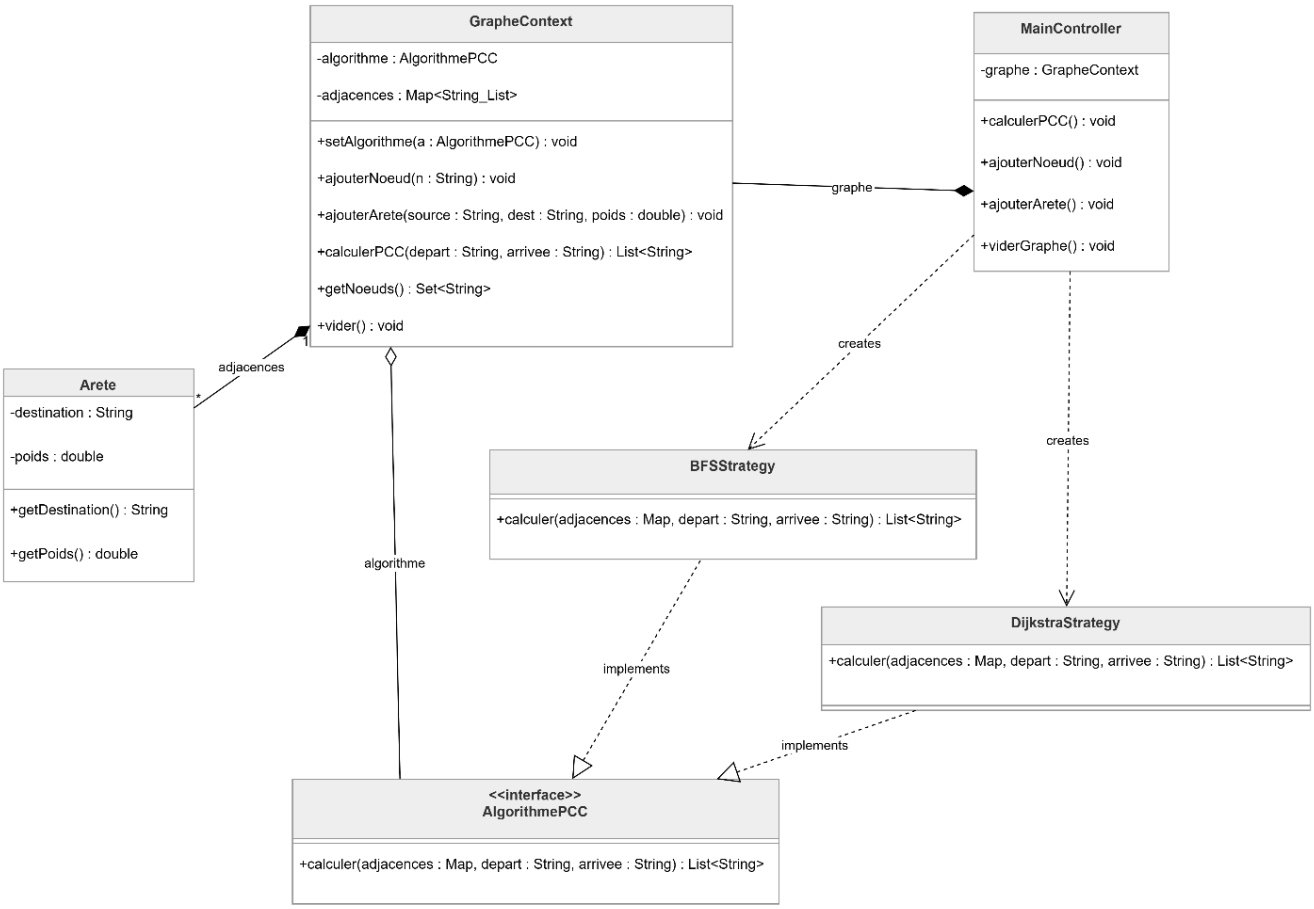


Figure 4: Diagramme Strategy — Algorithmes Plus Court Chemin (Dijkstra / BFS)

7. Patron 5 — Decorator

Contexte et problème résolu

On veut enrichir des formes existantes (ajouter une couleur personnalisée, une bordure épaisse) sans modifier les classes FormeRectangle, FormeCercle, FormeLigne et sans créer une explosion de sous-classes (RectangleRouge, RectangleBordureRouge, etc.).

> Le patron Decorator attache dynamiquement des responsabilités supplémentaires à un objet. Il offre une alternative flexible à la création de sous-classes pour étendre les fonctionnalités.

Diagramme UML

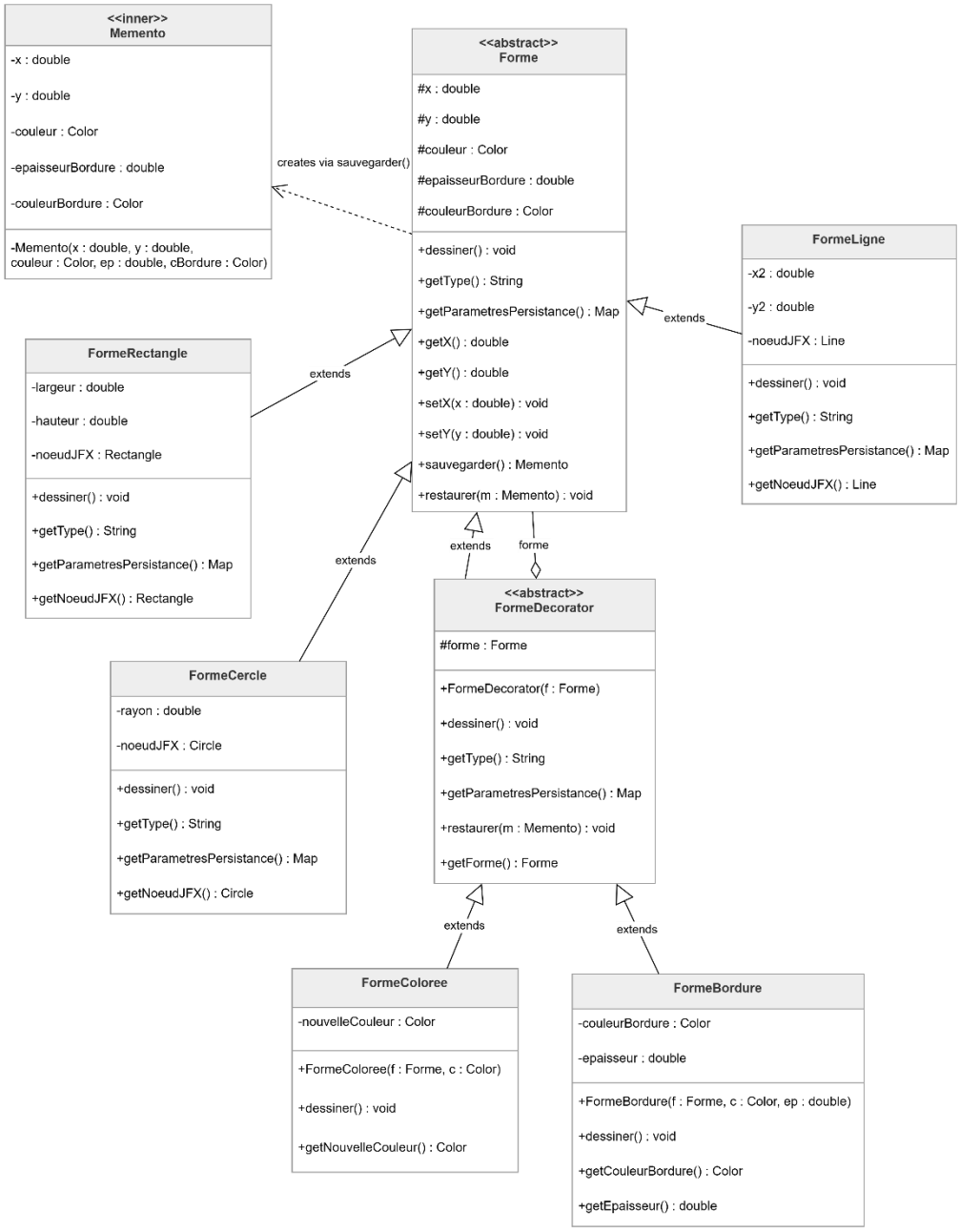


Figure 5: Diagramme Decorator — Enrichissement dynamique des formes

8. Patron 6 — Adapter

Contexte et problème résolu

DrawingCanvas doit opérer sur des formes sans dépendre directement de la hiérarchie concrète de Forme. L'interface FormeMetier définit un contrat métier minimal (dessiner, getX, getY, getType) indépendant des nœuds JavaFX. FormeJFXAdapter traduit ce contrat vers Forme.

> Le patron Adapter convertit l'interface d'une classe en une autre interface attendue par le client. Il permet à des classes incompatibles de travailler ensemble.

Diagramme UML

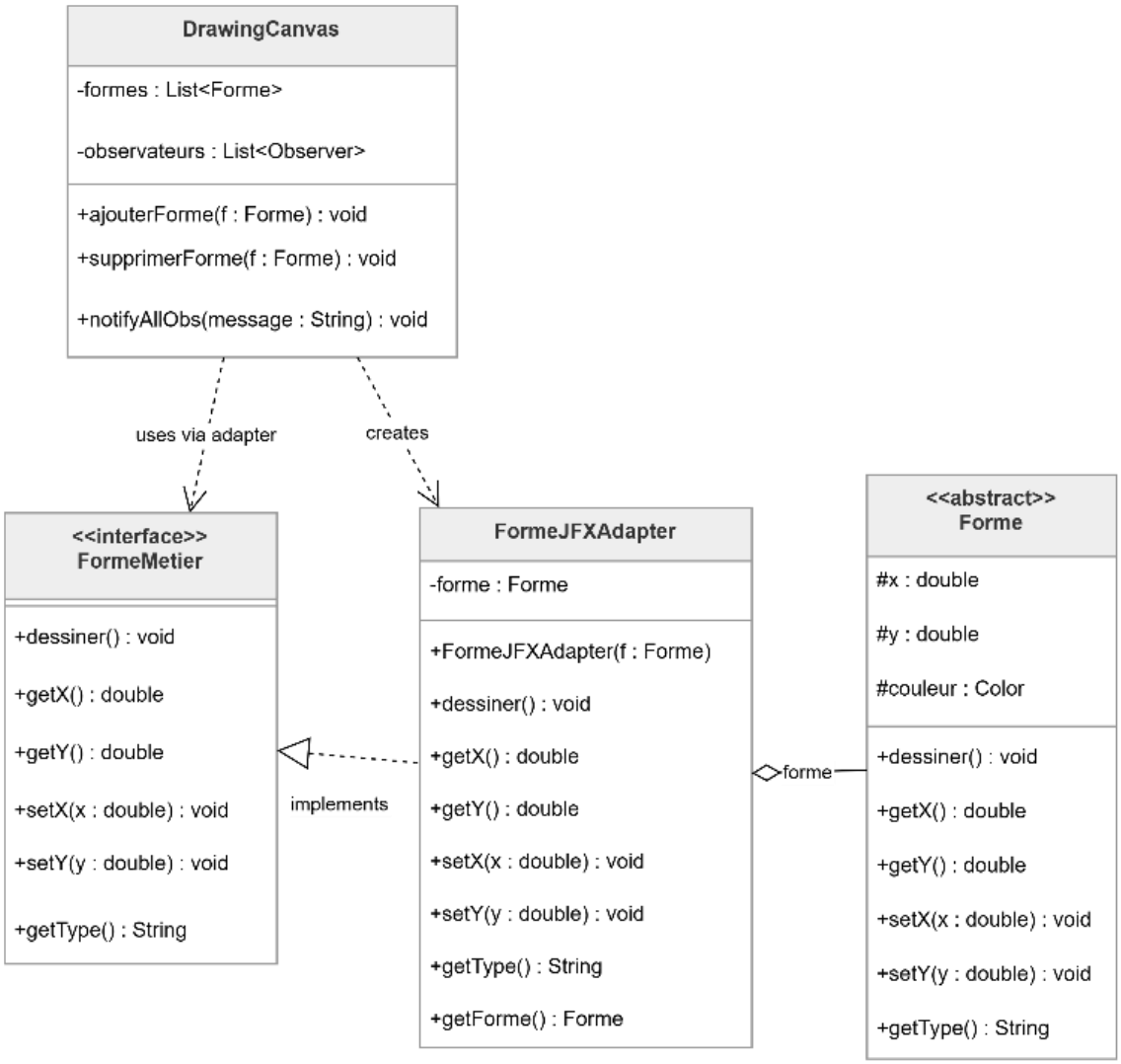


Figure 6: Diagramme Adapter — Découplage DrawingCanvas / hiérarchie Forme JavaFX

9. Patron 7 — Factory Method

Contexte et problème résolu

Lorsque l'utilisateur fait glisser la souris, l'application doit créer un FormeRectangle, un FormeCercle ou une FormeLigne selon le bouton sélectionné. Sans Factory, MainController contiendrait des if (type == RECTANGLE) new FormeRectangle(...) partout, couplant le contrôleur aux classes concrètes.

> Le patron Factory Method définit une interface pour créer un objet, mais laisse les sous-classes décider quelle classe instancier. Le client ne connaît que l'abstraction.

Diagramme UML

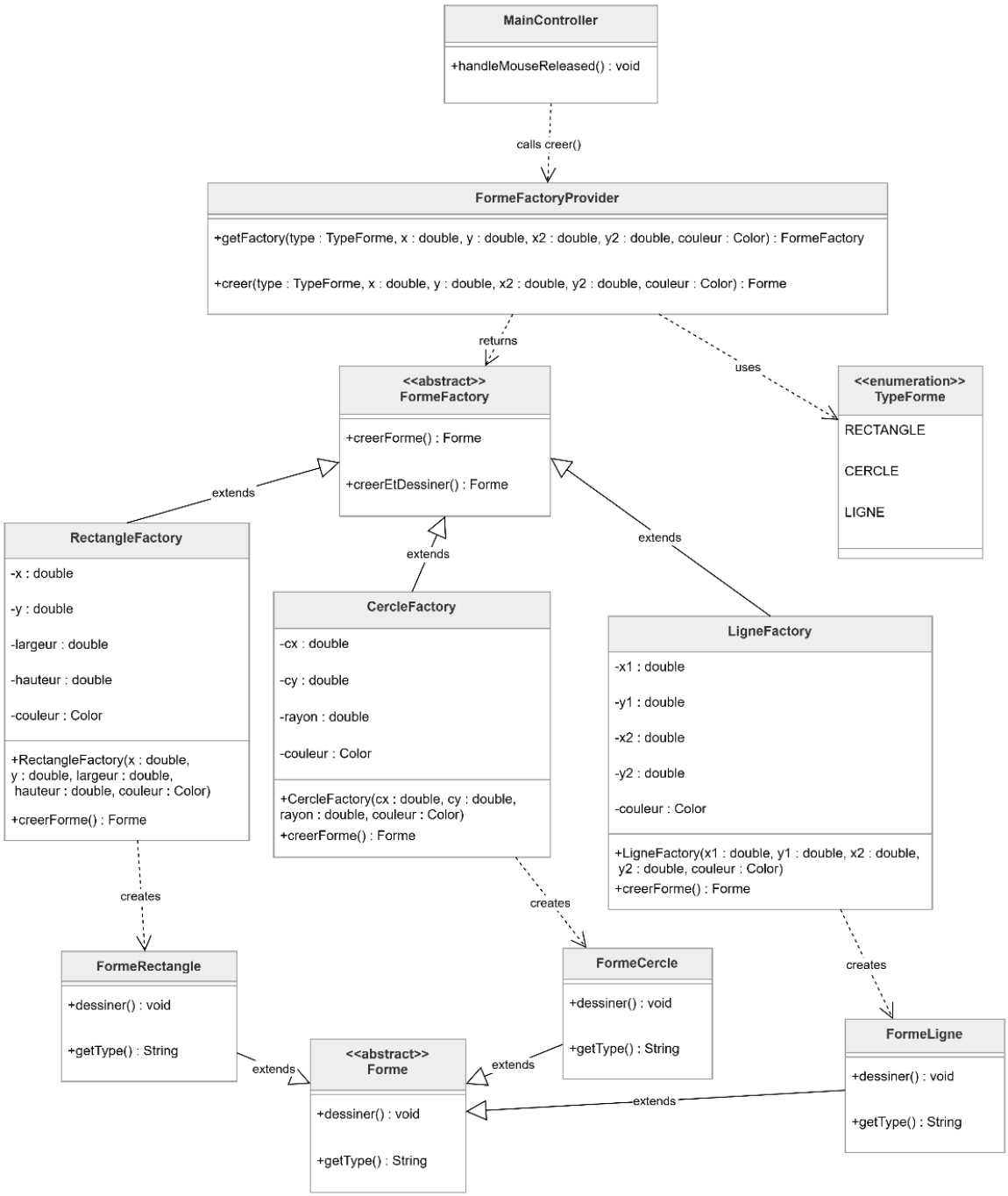


Figure 7: Diagramme Factory Method — Création de formes géométriques

10. Patron 8 — Command

Contexte et problème résolu

L'utilisateur doit pouvoir annuler et rétablir ses actions (Undo/Redo). Sans Command, il faudrait mémoriser l'état complet du canvas à chaque action, ou coder un mécanisme complexe dans le contrôleur. Avec Command, chaque action est encapsulée dans un objet avec execute() et undo().

> Le patron Command encapsule une requête en tant qu'objet, permettant ainsi de paramétrer des clients avec différentes requêtes, mettre en file d'attente des requêtes, et supporter les opérations annulables.

Diagramme UML

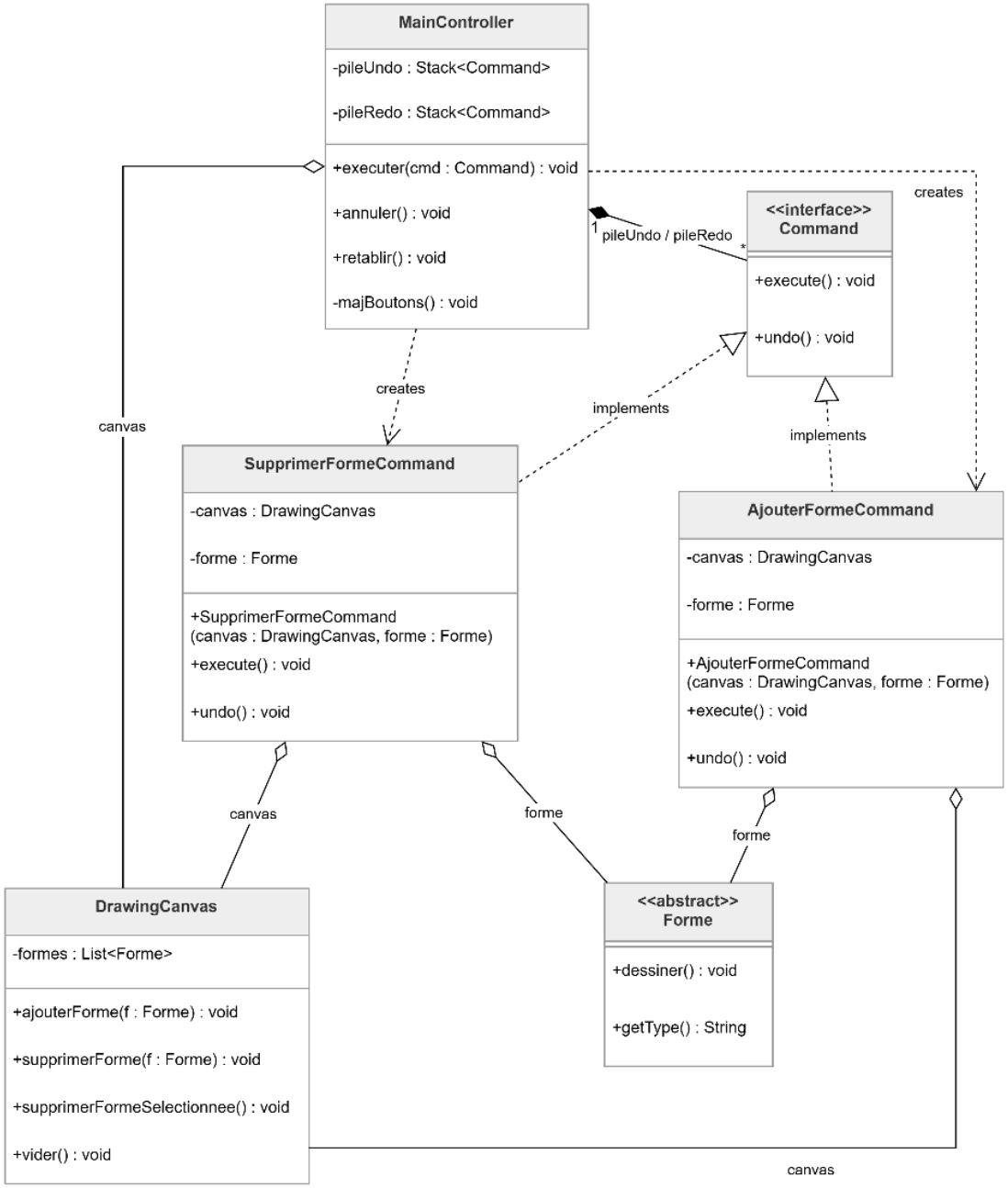


Figure 8: Diagramme Command — Undo / Redo des actions de dessin

11. Patron 9 — Memento

Contexte et problème résolu

Le patron Command gère le undo/redo structurel (ajout/suppression). Le patron Memento permet de capturer et de restaurer l'état interne d'une forme (position, couleur, bordure) sans violer l'encapsulation. Cela permet, par exemple, de déplacer une forme et de revenir à sa position précédente.

> Le patron Memento capture et externalise l'état interne d'un objet sans violer l'encapsulation, permettant de restaurer cet objet dans un état antérieur.

Diagramme UML

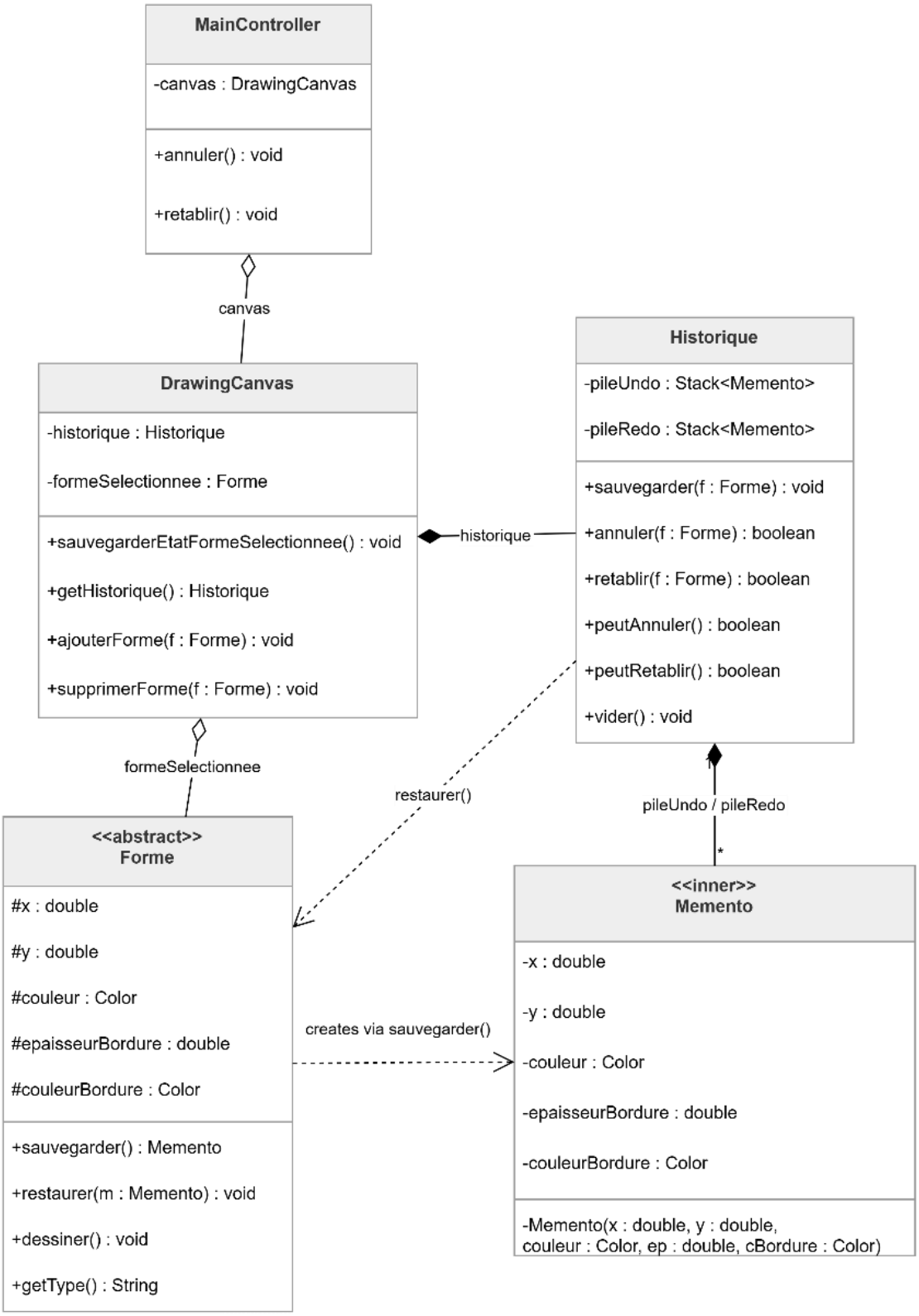


Figure 9: Diagramme Memento — Sauvegarde et restauration de l'état des formes

12. Synthèse et Tableau Récapitulatif

N°	Patron	Catégorie	Fonctionnalité couverte
1	Singleton	Création	Connexion BD unique et Logger centralisé
2	Observer	Comportement	Notification IHM et journalisation sur événements
3	Strategy	Comportement	Journalisation interchangeable (console / fichier / BD)
4	Strategy	Comportement	Algorithme PCC interchangeable (Dijkstra / BFS)
5	Decorator	Structure	Enrichissement dynamique des formes (couleur, bordure)
6	Adapter	Structure	Découplage DrawingCanvas / hiérarchie Forme JavaFX
7	Factory Method	Création	Création de formes sans couplage aux classes concrètes
8	Command	Comportement	Undo / Redo des actions de dessin et de suppression
9	Memento	Comportement	Sauvegarde / restauration de l'état interne des formes